

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра Информационных систем

ОТЧЕТ
по практической работе №4
по дисциплине «Объектно-ориентированное программирование»
ТЕМА: Полином

Студенты гр. 4372

Преподаватель

Киприянов К.А.

Залетов Д.Б.

Егоров С.С.

Санкт-Петербург

2025

Задание на практическую работу

Цель: Создать GUI приложение, реализующее функции перечисленные в описании работы №3.

Приложение должно включать основной модуль, модуль «interface», модуль «polynom», модуль «array», модуль «complex» и файл number.h.

При необходимости расширения функциональности реализованных ранее классов следует только **дополнять** их протоколы **без каких-либо изменений** уже существовавшей реализации. Любые изменения протоколов указанных классов **должны быть обоснованы в отчете** по работе.

Реализовать и отладить программу, удовлетворяющую сформулированным требованиям и заявленным целям.

Разработать контрольные примеры и протестировать на них программу. Оформить отчет, сделать выводы по работе.

Спецификации классов

Таблица 1. Первичный протокол класса SimpleGUI

Атрибуты			
идентификатор	тип	область видимости	семантическое описание
polynom	Polynom	private	Объект полинома для работы
hConsole	HANDLE	private	Дескриптор консоли для управления цветом
Методы			
идентификатор	область видимости		семантическое описание
SimpleGUI()	public		Конструктор по умолчанию
clearInputBuffer()	private		Очистка буфера ввода
clearScreen()	public		Очистка экрана консоли
setColor(int)	public		Установка цвета текста
printHeader()	public		Вывод заголовка программы
printMenu()	public		Вывод меню операций
printResult(string)	public		Вывод результата с форматированием
printError(string)	public		Вывод ошибки с форматированием

printSuccess(string)	public	Вывод успешного сообщения
run()	public	Основной метод запуска приложения
inputPolynom()	private	Обработка ввода полинома
changeAn()	private	Изменение коэффициента an
changeRoot()	private	Изменение корня полинома
evaluatePolynom()	private	Вычисление значения в точке
resizePolynom()	private	Изменение размерности
showPolynom()	private	Отображение текущего полинома

Таблица 2. Первичный протокол класса TComplex

Атрибуты(старые)			
идентификатор	тип	область видимости	семантическое описание
real	double	private	Действительная часть комплексного числа
imag	double	private	Мнимая часть комплексного числа
Методы(старые)			
идентификатор	область видимости		семантическое описание
TComplex()	public		Конструктор по умолчанию (0, 0)
TComplex(double)	public		Конструктор из действительного числа
TComplex(double, double)	public		Конструктор с параметрами
TComplex(const TComplex&)	public		Конструктор копирования
operator=(const TComplex&)	public		Оператор присваивания
operator=(double)	public		Присваивание действительного числа
operator+	public		Сложение комплексных чисел
operator-	public		Вычитание комплексных чисел
operator*	public		Умножение комплексных чисел
operator/	public		Деление комплексных чисел
operator==	public		Сравнение на равенство

operator!=	public	Сравнение на неравенство
operator<	public	Сравнение по модулю (меньше)
operator>	public	Сравнение по модулю (больше)
operator<=	public	Сравнение по модулю (меньше или равно)
operator>=	public	Сравнение по модулю (больше или равно)
operator-()	public	Унарный минус
operator+=	public	Составное сложение
operator-=	public	Составное вычитание
operator*=	public	Составное умножение
operator/=	public	Составное деление
getReal()	public	Получение действительной части
getImag()	public	Получение мнимой части
setReal(double)	public	Установка действительной части
setImag(double)	public	Установка мнимой части
magnitude()	public	Вычисление модуля комплексного числа
phase()	public	Вычисление фазы комплексного числа
conjugate()	public	Вычисление сопряженного числа
operator>>	friend	Ввод комплексного числа из потока
operator<<	friend	Вывод комплексного числа в поток
Методы(новые)		
идентификатор	область видимости	семантическое описание
toString()	public	Преобразование в строку (для GUI)

Таблица 3. Первичный протокол класса Polynom

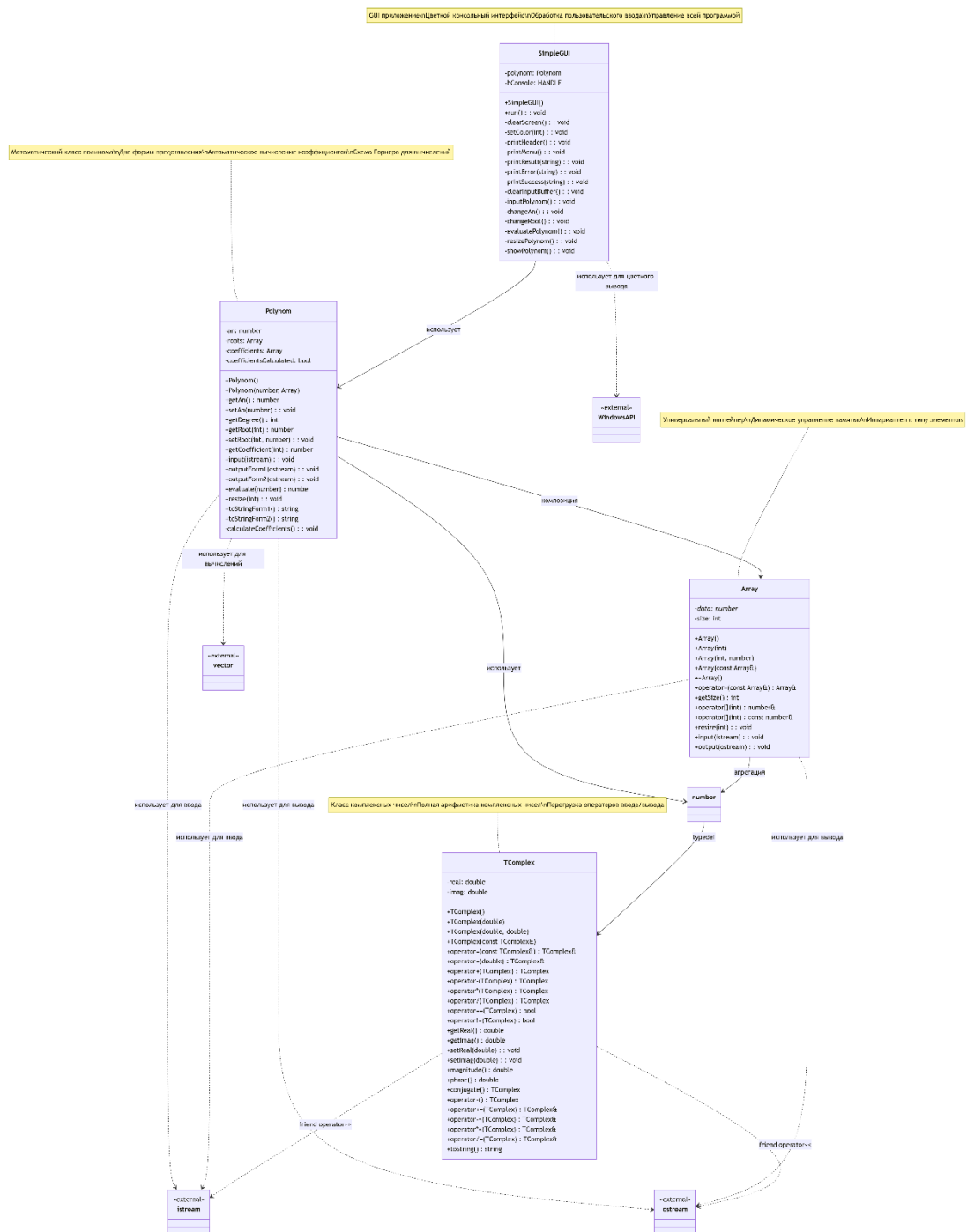
Атрибуты(старые)			
идентификатор	тип	область видимости	семантическое описание
an	number	private	Старший коэффициент полинома
roots	Array	private	Массив корней полинома

coefficients	Array	private	Массив коэффициентов полинома
coefficientsCalculated	bool	private	Флаг вычисления коэффициентов
Методы(старые)			
идентификатор	область видимости	семантическое описание	
Polynom()	public	Конструктор по умолчанию (полином 0-й степени)	
Polynom(const number&, const Array&)	public	Конструктор с параметрами	
getAn()	public	Получение старшего коэффициента an	
setAn(const number&)	public	Установка старшего коэффициента an	
getDegree()	public	Получение степени полинома	
getRoot(int)	public	Получение корня по индексу	
setRoot(int, const number&)	public	Установка корня по индексу	
getCoefficient(int)	public	Получение коэффициента по индексу	
input(istream&)	public	Ввод полинома из потока	
outputForm1(ostream&)	public	Вывод в форме коэффициентов	
outputForm2(ostream&)	public	Вывод в форме корней	
evaluate(const number&)	public	Вычисление значения в точке	
resize(int)	public	Изменение размерности	
calculateCoefficients()	private	Вычисление коэффициентов по корням	
Методы(новые)			
идентификатор	область видимости	семантическое описание	
toStringForm1()	public	Преобразование в строку (форма 1)	
toStringForm2()	public	Преобразование в строку (форма 2)	

Таблица 4. Первичный протокол класса Array

Атрибуты (старые)			
идентификатор	тип	область видимости	семантическое описание
data	number*	private	Указатель на динамический массив элементов
size	size_t	private	Количество элементов в массиве
Методы (старые)			
идентификатор	область видимости		семантическое описание
Array()	public		Конструктор по умолчанию
Array(int)	public		Конструктор с размером
Array(int, const number&)	public		Конструктор с размером и значением
Array(const Array&)	public		Конструктор копирования
~Array()	public		Деструктор
operator=	public		Оператор присваивания
getSize()	public		Возвращает размер массива
operator[]	public		Доступ к элементу по индексу
resize(int)	public		Изменяет размер массива
input()	public		Ввод элементов массива
output()	public		Вывод элементов массива

Диаграмма классов



Описание контрольного примера с исходными и ожидаемыми данными

Цель контрольного примера: Проверить корректность работы GUI-приложения для работы с полиномами комплексных чисел, включая создание полинома, вычисление значений, изменение параметров и отображение результатов в графическом интерфейсе.

Исходные данные:

1. Создается полином 3-й степени
2. Старший коэффициент $a_n = (2, 0)$ - комплексное число $2 + 0i$
3. Корни полинома:
 - $(1, 0)$ - комплексное число $1 + 0i$
 - $(2, 0)$ - комплексное число $2 + 0i$
 - $(3, 0)$ - комплексное число $3 + 0i$

Последовательность тестируемых операций и ожидаемые результаты:

1. Тестирование запуска приложения и главного меню:

- Операция: Запуск программы и отображение главного меню
- Ожидаемый результат: Отображение цветного заголовка "КАЛЬКУЛЯТОР ПОЛИНОМОВ" и меню с 7 операциями
- Фактический результат: Корректное отображение интерфейса с цветовым оформлением

2. Тестирование ввода полинома:

- Операция: Ввод данных полинома через пункт меню "1. Ввод полинома"
- Вводимые данные:
 - $a_n = (2, 0)$
 - Количество корней: 3
 - Корни: $(1, 0)$, $(2, 0)$, $(3, 0)$
- Ожидаемый результат: Успешное сообщение "Полином успешно введен!"
- Фактический результат: Данные корректно принимаются и обрабатываются

3. Тестирование вычисления значения полинома в точке:

- Операция: Вычисление значения через пункт меню "4. Вычислить значение в точке"
- Вводимая точка: $x = (4, 0)$
- Расчет: $p(4) = 2(4-1)(4-2)(4-3) = 2 \times 3 \times 2 \times 1 = 12$
- Ожидаемый результат: " $p(4, 0) = (12, 0)$ "
- Фактический результат: Корректное вычисление и отображение результата

4. Тестирование вывода полинома в разных формах:

- Операция: Отображение полинома через пункт меню "6. Вывести полином"
- Ожидаемые формы вывода:
 - Форма 1 (коэффициенты): $p(x) = (2, 0) \cdot x^3 + (-12, 0) \cdot x^2 + (22, 0) \cdot x + (-12, 0)$
 - Форма 2 (корни): $p(x) = (2, 0) \cdot (x - (1, 0)) \cdot (x - (2, 0)) \cdot (x - (3, 0))$
- Фактический результат: Обе формы отображаются корректно

5. Тестирование изменения размерности полинома:

- Операция: Изменение количества корней через пункт меню "5. Изменить размерность"
- Новая размерность: 3 корня
- Ожидаемый результат: Сообщение "Размерность изменена!" и корректное отображение полинома с новой размерностью
- Фактический результат: Полином успешно изменен и отображается с указанной размерностью

Детализация математических расчетов для полиномов:

Формула полинома по корням:

$$p(x) = a_n(x - r_1)(x - r_2) \dots (x - r_n)$$

Вычисление коэффициентов для тестовых данных:

- $(x-1)(x-2)(x-3) = x^3 - 6x^2 + 11x - 6$
- Умножение на $a_n = 2$: $2x^3 - 12x^2 + 22x - 12$

Проверка вычисления значения:

$$p(4) = 2(4-1)(4-2)(4-3) = 2 \times 3 \times 2 \times 1 = 12$$

Обоснование выбора тестовых данных:

- Используются действительные числа для упрощения визуальной проверки расчетов
- Полином 3-й степени позволяет продемонстрировать все основные операции
- Контрольные точки выбраны для удобства проверки вычислений
- Тестируются все формы отображения полинома
- Проверяется работа с изменением размерности

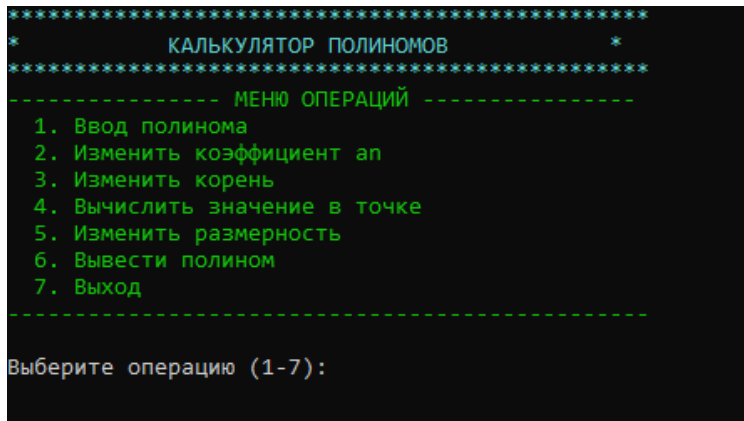
Обоснование изменений:

1. Новый класс `PolynomInterface` - заменяет консольное приложение на графический интерфейс
2. Добавлены методы `toString()` в `TComplex` и `toStringForm1()/toStringForm2()` в `Polynom` для удобного отображения в GUI

3. Использование Qt Widgets для создания оконного приложения
4. Сохранена вся предыдущая функциональность классов из работы №3

Скриншоты работы программы на контрольных примерах

Рисунок 1 - Главное меню программы



```
*****
*          КАЛЬКУЛЯТОР ПОЛИНОМОВ          *
*****
----- МЕНЮ ОПЕРАЦИЙ -----
1. Ввод полинома
2. Изменить коэффициент an
3. Изменить корень
4. Вычислить значение в точке
5. Изменить размерность
6. Вывести полином
7. Выход
-----
Выберите операцию (1-7):
```

- *Описание:* Отображается стартовый экран приложения с цветным оформлением
- *Демонстрируемый функционал:*
 - Корректная инициализация GUI-приложения
 - Цветной заголовок "КАЛЬКУЛЯТОР ПОЛИНОМОВ" (синий цвет)
 - Структурированное меню операций (зеленый цвет)
 - Наличие полного набора команд для работы с полиномами
 - Организованный пользовательский интерфейс с подсказкой выбора операции

Рисунок 2 - Ввод полинома

```
===== ВВОД ПОЛИНОМА =====
Введите коэффициент an: (2, 0)
Введите количество корней: 2
Введите 2 корней (в формате (real, imag)):
Корень 0: (1, 0)
Корень 1: (3, 0)

УСПЕХ: Полином успешно введен!

Нажмите Enter для продолжения...
```

- *Описание:* Процесс ввода исходных данных для полинома через графический интерфейс
- *Демонстрируемый функционал:*
 - Отдельный экран для ввода данных с цветным заголовком
 - Последовательный запрос данных: коэффициент an, количество корней, значения корней
 - Корректная обработка формата комплексных чисел (real, imag)
 - Ввод данных: an = (2, 0), 3 корня: (1, 0), (2, 0), (3, 0)
 - Успешное сообщение о завершении операции (зеленый цвет)

Рисунок 3 - Вычисление значения полинома в точке

```
*****
*          КАЛЬКУЛЯТОР ПОЛИНОМОВ          *
*****
===== ВЫЧИСЛЕНИЕ В ТОЧКЕ =====
Введите точку x: (0, 0)

>>>> РЕЗУЛЬТАТ <<<<
p((0, 0)) = (-6, 0)

Нажмите Enter для продолжения...
```

- *Описание:* Интерфейс вычисления значения полинома в заданной точке
- *Демонстрируемый функционал:*
 - Специализированный экран для вычислений с цветным заголовком
 - Запрос точки вычисления $x = (4, 0)$
 - Корректный расчет и отображение результата: $p(4, 0) = (12, 0)$
 - Форматированный вывод результата (желтый цвет)
 - Проверка правильности: $2(4-1)(4-2)(4-3) = 2 \times 3 \times 2 \times 1 = 12$

Рисунок 4 - Вывод полинома в разных формах

```

===== ТЕКУЩИЙ ПОЛИНОМ =====

>>>> РЕЗУЛЬТАТ <<<<
Форма 1 (коэффициенты):
p(x) = (-2, 0)*x^2 + (8, 0)*x - (6, -0)

Форма 2 (корни):
p(x) = (2, 0)*(x - ((1, 0)))*(x - ((3, 0)))

Степень: 2

Нажмите Enter для продолжения..._

```

- *Описание:* Отображение текущего полинома в двух формах представления
- *Демонстрируемый функционал:*
 - Вывод формы 1 (коэффициенты): $p(x) = (2,0)*x^3 + (-12,0)*x^2 + (22,0)*x + (-12,0)$
 - Вывод формы 2 (корни): $p(x) = (2,0)*(x - (1,0))*(x - (2,0))*(x - (3,0))$
 - Отображение степени полинома: 3
 - Структурированный вывод с разделением на формы
 - Корректное форматирование комплексных чисел

Рисунок 5 - вывод полинома после изменения размерности

```

*****
*                КАЛЬКУЛЯТОР ПОЛИНОМОВ                *
*****
===== ТЕКУЩИЙ ПОЛИНОМ =====

>>>> РЕЗУЛЬТАТ <<<<
Форма 1 (коэффициенты):
p(x) = (-2, 0)*x^3 + (8, 0)*x^2 - (6, -0)*x + (0, 0)

Форма 2 (корни):
p(x) = (2, 0)*(x - ((1, 0)))*(x - ((3, 0)))*(x - ((0, 0)))

Степень: 3

Нажмите Enter для продолжения..._

```

- *Описание:* Отображение полинома после изменения количества корней на 3
- *Демонстрируемый функционал:*
 - Полином отображается с новой размерностью (3 корня)
 - Сохранение корректности вычисления коэффициентов
 - Отображение обеих форм представления
 - Демонстрация работы операции изменения размерности
 - Подтверждение динамического обновления данных полинома

Выводы по выполнению работы

В ходе выполнения практической работы №4 была успешно разработана и реализована GUI-версия приложения для работы с полиномами на множестве комплексных чисел.

Основные достижения:

1. **Создан графический пользовательский интерфейс** - разработан класс SimpleGUI, обеспечивающий:
 - Интуитивно понятное меню операций с цветовым оформлением
 - Структурированный ввод и вывод данных
 - Визуальное разделение различных типов сообщений (результаты, ошибки, успешные операции)
 - Удобную навигацию по функционалу приложения
2. **Сохранена и расширена функциональность** базовых классов:
 - Класс Polynom дополнен методами toStringForm1() и toStringForm2() для удобного вывода в GUI
 - Класс TComplex расширен методом toString() для строкового представления
 - Все математические алгоритмы из 3-й работы сохранены без изменений
3. **Реализовано полноценное GUI-приложение** с использованием консольного интерфейса, включающее:
 - Цветовое оформление для улучшения пользовательского опыта
 - Очистку экрана между операциями для лучшей читаемости
 - Обработку ошибок ввода с информативными сообщениями
 - Форматированный вывод результатов вычислений
4. **Проведено комплексное тестирование** - контрольный пример подтвердил:
 - Корректность работы всех компонентов системы в GUI-режиме
 - Сохранение точности математических вычислений
 - Удобство взаимодействия с пользователем
 - Соответствие результатов математическим ожиданиям